GREENROOM · SUPPORTING DOCUMENT · IMPLEMENTATION & INSIGHT REFERENCE · MAY 2026

Features, APIs, Tabs, and the Insights Behind the Redesign

This document is the engineering and analytical companion to the Settlement Redesign report. It walks every tab, every API endpoint, and every backend module in the live Greenroom build, and — for each one — names the data it surfaces and the role it plays in the redesign argument. Read the redesign report for the conclusion; read this for the receipts.

Sourced directly from the live codebase at this commit (`artifacts/api-server`, `artifacts/greenroom`) and the live API at `http://localhost:8080/api`. 24-month window unless otherwise noted.

Contents

1. Stack & architecture decisions

LAYER	CHOICE
Monorepo	pnpm workspaces, Node.js 24, TypeScript 5.9
API server	Express 5, esbuild bundle (CJS) — restart workflow after backend edits, HMR does not apply
Database	libsql/sqlite, file-backed at `artifacts/api-server/data/greenroom.db`, Drizzle ORM
Frontend	Vite + React + wouter + Tailwind v4 + shadcn/ui + Recharts
Validation	Zod (`zod/v4`) + drizzle-zod
API codegen	Orval, from OpenAPI spec in `@workspace/api-spec`
LLM	

Provider-agnostic helper (`lib/llm.ts`) wrapping Anthropic + OpenAI SDKs; provider/key/model configurable at runtime via the Settings tab

KEY ARCHITECTURAL DECISIONS

- **No drizzle-kit for the Greenroom DB.** The artifact's data is a local libsql file, not workspace Postgres. Schema changes are applied at boot via small `ensureColumn` / `CREATE TABLE IF NOT EXISTS` helpers in `db/index.ts`. Keeps the artifact self-contained and trivially seedable from a snapshot.
- **All LLM access goes through one helper.** `lib/llm.ts` is the only module that imports the Anthropic or OpenAI SDKs. Call sites only see `llmGenerateText` / `llmGenerateJson` / `llmIsConfigured`. This is what lets the Settings tab swap providers globally with a single save.
- **Settings keys are write-only across the API.** The settings GET/POST responses contain only `{configured, source, model}` per provider — raw API keys are never shipped back to the client.
- **Insights cache uses a race-safe pending promise.** First request computes; concurrent requests await the same in-flight promise; cache invalidates on server restart, Settings save, or explicit `clearInsightsCache()`.
- **Smart Switch owns flat conversion; Improve Deal is caps-only.** The April 2026 audit showed "convert this %-deal to a flat" is only data-safe in cells Smart Switch already covers (`vs` / `pn` at $1–5K, `door` any size). Improve Deal therefore emits only structural-cap suggestions (expense cap, hospitality cap), with audit-derived P75-flat defaults.

2. Data model

SQLite tables in `artifacts/api-server/src/db/schema.ts`. Asterisked fields were added by runtime `ensureColumn` migrations as the redesign work surfaced new requirements.

TABLE	PURPOSE	KEY FIELDS
`artists`	Artist roster	id, name
`agents` / `agencies`	Agency lookup for booking provenance	id, name, agencyId
`venues`	Venue list (single venue today: The Crescent)	id, name
`shows`	One row per scheduled or completed show	id, artistId, agentId, venueId, date, status
`deals`	Pre-show financial structure	showId, dealType, guaranteeAmount, percentage, expenseCap*, hospitalityCap*, bonusesJson*, dealNotesFreetext*
`settlements`	Post-show reconciliation	showId, status, grossBoxOffice, totalToArtist, recoupsJson, notes,

		signoffText, disputedAt, positiveSummary*, negativeSummary*
`ticket_sales` / `expenses` / `comps` / `recoups`	Per-show line items joined into the show detail view and the settlement wizard	showId, category, amount, count, faceValue
`guarantee_suggestions`	Smart Guaranteed Price (SGP) results, one per upcoming show	showId, suggested, tier, expectedGrossSource, expenseSource, winner, basis, stepsJson
`switch_suggestions`	Smart Switch results — proposed alternative deal shape	showId, source, tier, suggestedDealType, projectedToArtist, basis, status (proposed/accepted/declined)
`settings`	k/v store for LLM provider/key/model	key, value

Idempotent seed snapshot lives at `artifacts/api-server/data/seeds/` as JSON exports plus a byte-for-byte copy of the live `greenroom.db`. Refresh recipe: `pnpm --filter @workspace/api-server exec tsx scripts/exportSeedsSnapshot.ts`.

3. The seven tabs

The sidebar (`components/layout/nav-links.tsx`) exposes seven tabs. For each, this section names what the page shows, where the numbers come from, and what the page contributed to the redesign argument.

3.1 Shows · `/shows` & `/shows/:id`

What it does. Lists every past or current show (`shows.date ⩽ today`) as a card grid with deal-type / size-bucket / attention filters. Detail page joins the show against artist, agent, agency, venue, deal, settlement, ticket sales, expenses, comps, and recoups; each show also has a **Settle** wizard at `/shows/:id/settle`.

Calculated flags. `isUnsupportedDeal` = `dealType ∈ {percentage_of_net, vs, door}` (the three the in-app wizard cannot model). `isDisputed` = settlement status disputed OR any recoup line in `recoupsJson` has `status ⩵ "disputed"`.

Why it mattered for the redesign. The "unsupported deal" badge across the card grid was the first hint that the wizard's coverage was structurally limited — every flagged deal was a settlement-night spreadsheet, not a tool transaction. That observation set up the "what fraction of past deals could in principle be settled in-app?" KPI on the Reports tab (next).

3.2 Artists · `/artists`

What it does. Grid of every artist with show counts, last show date, top deal type, review-tone topics drawn from LLM summaries, and a flagged-attention badge. Numbers from `getAllArtists` in `lib/queries.ts`.

- **showCount / lastShowDate** — `count(shows.id)`, `max(shows.date)` grouped by artist.
- **topDealType / dealTypes[]** — counts of each `deals.dealType` per artist via `shows`; highest-count is the headline chip.
- **topPositive / topNegative** — first sentence (≤90 chars) of the most-recent non-empty `settlements.positiveSummary` / `negativeSummary`; populated by the LLM enrichment pipeline.
- **attentionCount** — number of `getNeedsAttention()` items whose `showId` belongs to this artist.

Why it mattered. Surface-level test for whether qualitative friction clusters by artist (loud-booker effect) or by deal shape. Result: clusters by deal shape, not artist — which made the cell-grid the right unit of analysis on the Insights tab.

3.3 Reports · `/reports`

What it does. High-level KPIs across all past shows. Numbers from `getReports`.

- **dealTypeCounts** — distribution of `deals.dealType`.
- **inAppToolUsageRate** = `count(dealType ∈ {flat, percentage_of_gross}) / total deals`. This is the headline number that opens the redesign report.
- **settlementStatus** + **disputedRate** = `disputed / totalSettlements`.
- **totalGross** / **totalToArtists** — sums across past settlements.
- **totalRecoupValue** / **disputedRecoupValue** / **settlementsWithRecoups** — parsed from `recoupsJson`.
- **totalCompTickets** / **totalCompFaceValue** / **compsByCategory**.

Why it mattered. Established the size of the "out-of-tool" cohort — the share of bookings that escape the wizard entirely. That cohort is what Phases 1–2 of the redesign target.

3.4 Deal Analysis · `/deal-analysis`

What it does. Quantitative breakdown of what kinds of deals the venue runs and how they perform. Powered by `getDealAnalysis` in `lib/queries.ts`.

- **byComplexity** — every past deal classified `simple` / `medium` / `complex` by `classifyComplexity()`. Per bucket: `count`, `pct`, `avgPayout`, `inToolCount`, `spreadsheetCount`.
- **bySize** — bucketed by guarantee into `$0-1K`, `$1-5K`, `$5-15K`, `$15K+`, plus an `Uncapped %` bucket for guarantee-less percentage deals. Per bucket: `count`, `pct`, `avgGross`, `avgToArtist`, `disputeRate`, `losingMoneyCount`, `profitN`.

- **byProfitability** — partition every settled show with both gross and payout into `profitable` / `unprofitable` using `gross - totalToArtist - Σ expenses`; reports per-side `disputeRate`.
- **costs** — totalExpenses, expensesByCategory, totalRecoups, disputedRecoupValue, recoupsByCategory.
- **revenue.byDealType** — per `dealType`: Σ gross, Σ netToVenue, Σ toArtist, settlement count.
- **revenue.months** — last 24 calendar months: gross, netToVenue, toArtist, plus `byType` for stacked-area charts.
- **revenue.crossTabBySizeAndType** — the cross-tab grid (rows = deal type, columns = size bucket). Per cell: `count`, `settledN`, `profitN`, `losingMoneyCount`, `disputed`, `losingMoneyRate`, `disputeRate`, `attentionCount`, `attentionRate`, `attentionByKind`.

Why it mattered. The cross-tab is the analytical backbone of the redesign. Every claim about "vs deals at $1–5K dispute at 12.4% versus 3.8% on flats at the same size" comes from a single cell of this grid. The cells also gave Smart Switch its bucket-aware policy.

3.5 Needs Attention · `/needs-attention`

What it does. A worklist of past shows whose data smells off. `getNeedsAttention()` emits four rule kinds:

- `show_settled_no_settlement` — show status is `settled` / `closed` but there's no settlements row.
- `notes_say_closed_but_status_open` — settlement is not in a closed status but notes/signoffText contain a closure phrase from a fixed vocabulary (closed out, settled up, paid in full, signed off, …). The matched phrase becomes `evidence`.
- `disputed_recoups_but_signed` — settlement is closed but at least one recoup line in `recoupsJson` has `status === "disputed"`. Lists labels + amounts as evidence.
- `stale_disputed` — settlement status is disputed and `disputedAt` is more than 30 days ago.

Why it mattered. These four kinds were the empirical definition of "settlement friction" used by the LLM enrichment pipeline and the Insights cell grid. They also gave a deterministic dispute-rate denominator that doesn't depend on freetext interpretation.

3.6 Insights · `/insights`

What it does. The qualitative companion to Deal Analysis. Same `dealType × sizeBucket` grid, but each cell shows the dominant friction kind plus a top-5 cluster of recurring complaints. Powered by `getInsights()` and `enrichSettlements()` in `lib/insights.ts`.

1. **Per-settlement enrichment** (`POST /api/insights/enrich`). Picks every settlement that has notes, signoffText, a disputed status, or a disputed recoup. Calls the active LLM with a structured payload and stores `positiveSummary` + `negativeSummary` (1–2 sentences each, strict JSON output). Concurrency = 8, idempotent.
2. **Grid build** (`GET /api/insights`). Re-uses `classifySizeBucket()` from Deal Analysis. Per cell: `count`, `attentionCount`, `byKind[k]`, `topKind` / `topKindCount`. Ties broken by `KIND_PRIORITY = [stale_disputed, disputed_recoups_but_signed, show_settled_no_settlement, notes_say_closed_but_status_open]`.

3. **Complaint clustering** — for each cell with a `topKind`, gather the `negativeSummary` of every flagged deal in that cell, then call the LLM once per cell to cluster them into ≤5 themes `{theme, count}`, ordered by count desc.
4. **Caching** — full payload cached in-module after first compute, reused until server restart, Settings save, or explicit `clearInsightsCache()`. Concurrent requests await the same in-flight promise (`pending`).

Reports `enrichmentCoverage = withSummary / total` as a transparency signal (currently 237 / 558 ≈ 42%).

Page also hosts three live engines (added during the redesign work):

- **Switch Savings** (`GET /api/insights/switch-savings`) — historical cost of not having Smart Switch. Top-line tiles: cumulative money saved, time saved, and the new `vsPercentageFiredStats` (% of settled vs deals where the percentage clause out-paid the guarantee, plus avg guarantee-win when it didn't).
- **SGP Backtest** (`GET /api/insights/guarantee-backtest`) — replays each past deal with the Smart Guaranteed Price algorithm using only data available before the show. Top-line tiles: money protected, money overpaid, net delta, and the new `gapCoverage` distribution (median / p75 / p90 of |SGP - paid|, plus the share of gaps fully covered at thresholds $100/$200/$400/$800/$1,500).
- **Switch Projected Grid** (`GET /api/insights/switch-projected-grid`) — the cell-by-cell rollup of "if Smart Switch had been used, this cell would have paid X instead of Y." This is the source of the headline "$23,024 saved across 137 vs/$1–5K deals" claim in the redesign report.

3.7 Settings · `/settings`

Configures which LLM the server calls and stores keys persistently. UI: provider radio (Anthropic / OpenAI), per-provider API-key password input (`•••••••• (saved)` + Clear when persisted), per-provider model dropdown, save button, footer line showing active provider/model/source. Backend in `lib/llm.ts` + routes in `routes/greenroom.ts` — see §6 for full behaviour.

4. API surface

Every route is mounted under `/api` on the Express app in `artifacts/api-server/src/routes/greenroom.ts`.

METHOD · PATH	PURPOSE
`GET /shows`	Past show list with joined artist/agent/deal/settlement.
`GET /shows/:id`	Full show detail (also used by the Settle wizard).
`GET /shows/:id/export`	JSON export with per-show LLM deep summary.
`GET /artists`	Artist roster with deal-type and review-topic enrichment.
`GET /reports`	KPI bundle for the Reports tab.

`GET /deal-analysis`	Full Deal Analysis bundle (complexity / size / profitability / costs / revenue.byDealType / months / crossTabBySizeAndType).
`GET /needs-attention`	Flat list of attention items.
`POST /insights/enrich`	Run per-settlement summary pass; `{totalCandidates, enriched, skippedExisting, failed}`.
`GET /insights`	Cached cell grid with `topKind` + clustered complaint bubbles.
`GET /insights/switch-savings`	Top-N historical Smart Switch counterfactual savings + `vsPercentageFiredStats`.
`GET /insights/switch-projected-grid`	Cell rollup of projected Smart Switch payouts vs actual.
`GET /insights/guarantee-backtest`	SGP backtest items + `gapCoverage`.
`POST /guarantee/backfill`	Backfill SGP for upcoming shows (idempotent).
`POST /shows/:id/guarantee/generate`	Generate or recompute SGP for one show.
`POST /shows/:id/deal/apply-guarantee`	Apply the SGP suggestion to the show's deal.
`GET /shows/:id/deal/improvements`	Improve-Deal proposals (caps-only) for one show.
`POST /shows/:id/deal/apply-improvements`	Persist accepted Improve-Deal items into `deals`.
`POST /shows/:id/switch/generate`	Generate or recompute the Smart Switch suggestion for one show.
`POST /shows/:id/switch/accept`	Mark a switch suggestion accepted; rewrites `deals.dealType`.
`POST /shows/:id/switch/decline`	Mark a switch suggestion declined.
`GET /settings/llm`	Read-only LLM status (provider/model/source/`hasKey`); never returns key material.
`POST /settings/llm`	Upsert provider/key/model; clears the insights cache.

5. Backend lib modules

`lib/queries.ts`

Source of truth for everything except Insights. `getAllShows`, `getShowById`, `getAllArtists`, `getReports`,

`lib/insights.ts`

The enrichment + clustering pipeline. Two LLM call sites (per-settlement summary, per-cell complaint cluster), both gated on

`getDealAnalysis`, `getNeedsAttention`. Bucketing helpers `classifyComplexity()` and `classifySizeBucket()` are re-used by Smart Switch, the Backtest, and Insights — so every cell-grid in the app uses the same definition.

`llmIsConfigured()`. Owns the in-module cache and the `KIND_PRIORITY` tie-breaker.

`lib/smartGuarantee.ts`

Smart Guaranteed Price — a 7-step algorithm that proposes a flat guarantee for an upcoming show. Inputs: artist's prior payouts at this venue, expected gross (own past or cell mean), expense baseline. Outputs: `suggested`, `tier` (A–D confidence), `winner` (guarantee/percentage/tie), and a structured `stepsJson` for "show your work." Tier D when cell `n < 3`; first-time artists demote A→B.

`lib/smartSwitch.ts`

Proposes an alternative deal shape (flat or door-hybrid) for vs/pn/door deals. Five `SwitchSource` branches (`sgp_engine`, `guarantee_amount`, `cell_mean`, `door_hybrid_calc`, `door_dead_pool`, `suppressed`) so the UI can badge provenance. Tier A demoted to B when historical band > $1,000.

`lib/dealImprovements.ts`

Caps-only post-MVP "Improve Deal" engine. Emits structural-cap suggestions (expense cap, hospitality cap) with audit-derived P75-flat defaults. Deliberately does not propose flat conversion — that is Smart Switch's job in cells where the data is safe.

`lib/switchSavings.ts`

"If Smart Switch had been used, what would have changed?" Computes per-show counterfactual payout, money delta, and a notes-/recoup-derived time-saved estimate. `getSwitchProjectedGrid()` rolls the same logic up to the cell-grid for the redesign report. Now also reports `vsPercentageFiredStats`, scoped to the $1–5K bucket where Smart Switch's flat fallback applies: 11/64 (17.2%) of settled vs / $1–5K deals had the percentage clause never out-pay the guarantee.

`lib/guaranteeBacktest.ts`

Replays each past deal with the SGP algorithm using only data available before the show. Outputs per-deal `direction` (`money_protected` / `money_overpaid` / `even`) plus `moneyProtected`,

`lib/showExport.ts`

Per-show JSON export with an LLM deep summary. Routes through `llmGenerateText` with a heavier

`moneyOverpaid`, `netDelta`. Now also reports `gapCoverage` (median $1,119 · p75 $1,939 · p90 $3,344) used for Phase-3 Product 2 cap sizing.

`modelOverride` (`claude-sonnet-4-6` on Anthropic, `gpt-4o` on OpenAI).

`lib/llm.ts`

Single source of truth for which provider/key/model to call. `getLlmConfig()` reads the `settings` table on every call, falling back to `AI_INTEGRATIONS_*` env vars. `llmGenerateText` + `llmGenerateJson<T>` are the only two functions other modules import. `extractJson()` tolerantly strips ```json fences and falls back to the first `{…}` substring before `JSON.parse`.

`lib/logger.ts`

Pino-based structured logger; the API server bundles it into `dist/pino-*.mjs` via esbuild.

6. LLM helper & settings

- **Storage.** `settings (key TEXT PRIMARY KEY, value TEXT)` created on startup by `migrationsReady`. Keys: `llm.provider`, `llm.anthropic.apiKey`, `llm.anthropic.model`, `llm.openai.apiKey`, `llm.openai.model`.
- `getLlmStatus()` / `GET /api/settings/llm` returns metadata only: `activeProvider`, `activeModel`, `source ∈ {settings, env, none}`, `hasKey`, plus per-provider `{configured, source, model}` and the list of selectable models. Never returns raw key material.
- `saveLlmSettings()` / `POST /api/settings/llm` accepts partial updates (provider / *ApiKey / *Model). After saving, the insights cache is cleared so the next `GET /api/insights` re-runs against the new active model.
- `llmGenerateText({prompt, modelOverride?, maxTokens?})` — provider-agnostic text completion. Anthropic path: `messages.create` with active model; first text block of the response. OpenAI path: `chat.completions.create` with `messages: [{role:"user", content: prompt}]`; first `choice.message.content`.
- `llmGenerateJson<T>()` wraps text completion with the tolerant `extractJson` fence stripper.
- **BASE_URL fallback.** When a saved key is present, the env-var `BASE_URL` override is intentionally NOT applied, so user-supplied keys hit the canonical provider endpoint.

7. The insight pipeline that shaped the redesign

Each phase of the redesign argument is anchored to a specific data surface in the live app. The chain runs left-to-right from raw rows to a publishable claim.

CLAIM IN THE REDESIGN REPORT	SURFACE THAT SUPPLIES IT	MODULE / ROUTE
Only ~X% of past bookings could be settled in-app	Reports → `inAppToolUsageRate`	`queries.getReports`
vs/$1–5K disputes at 12.4% vs 3.8% on flat at the same size	Deal Analysis → `revenue.crossTabBySizeAndType` cell `disputeRate`	`queries.getDealAnalysis`
Switching the 137 vs/$1–5K deals to flat would have saved $23,024	Insights → Switch Projected Grid (`cell.payoutDelta`)	`switchSavings.getSwitchProjectedGrid`
The percentage clause out-pays the guarantee in 86.7% of vs deals (avg = $0 when it doesn't)	Insights → Switch Savings tile (`vsPercentageFiredStats`)	`switchSavings.getSwitchSavings`
Door deals are the single largest savings cell (~$50K via hybrid)	Insights → Switch Projected Grid for door rows	`switchSavings.getSwitchProjectedGrid`
		`guaranteeBacktest.getGuaranteeBacktest`

SGP suggestion-gap distribution (median $1,119 · p75 $1,939) used to size Product 2 cap	Insights → SGP Backtest tile (`gapCoverage`)	
4.3% headline dispute rate; intra-venue cell range 3.8% – 12.4% used as the Phase-3 platform sensitivity band	Reports → `disputedRate` + Deal Analysis cells	`queries.getReports` + `queries.getDealAnalysis`
Friction clusters by deal shape, not by artist	Insights → cell grid + per-cell complaint clusters	`insights.getInsights`
"Improve Deal is caps-only" policy	Audit on Deal Analysis cells: bucket-scaling unsafe outside Smart Switch's covered cells	`dealImprovements.getDealImprovements`

> **The pipeline is reproducible.** Every figure cited in the redesign report can be re-derived by hitting the live API at this commit. The `/insights` page renders all of them as on-screen tiles, and the seed snapshot in `artifacts/api-server/data/seeds/` reproduces the underlying database byte-for-byte.

8. Live reference numbers

Snapshot of the figures the redesign report cites, as returned by the live API at this commit. Re-run the curls below to verify.

QUANTITY	VALUE	SOURCE
Past shows in window	510	`GET /api/reports`

Past settlements	509	`GET /api/reports`
Disputed settlements	22 (4.3%)	`GET /api/reports` · `disputedRate`
Disputed-recoup value	$7,582 (avg $345)	`GET /api/reports`
vs · $1–5K past deals	137	`GET /api/deal-analysis` · cross-tab cell
vs · $1–5K dispute rate	12.4%	same cell · `disputeRate`
flat · $1–5K dispute rate	3.8%	same cell · `disputeRate`
Smart Switch projected savings · vs/$1–5K	$22,774	`GET /api/insights/switch-projected-grid`
vs deals scanned · % clause never fired (scoped to $1–5K)	11 / 64 (17.2%)	`GET /api/insights/switch-savings` · `vsPercentageFiredStats`
SGP backtest scored deals	321	`GET /api/insights/guarantee-backtest`
SGP gap distribution (median · p75 · p90)	$1,119 · $1,939 · $3,344	same · `gapCoverage`
SGP gap fully covered at $400	25.2%	same · `gapCoverage.buckets`
LLM enrichment coverage	237 / 558 (42%)	`GET /api/insights` · `enrichmentCoverage`

END · Greenroom Supporting Document · accompanies `greenroom-settlement-redesign-v2.pdf` (v2.1, May 2026).